

Task Debugging

Title: Electron App — Login Crash Debug Report

1. Document metadata

- Author: *Hiruni Ranska*
 - Date: 2025/11/14
 - Project: Electron LibreView (LLU) client
 - File name suggestion: Login_Crash_Debug_Report.docx
-

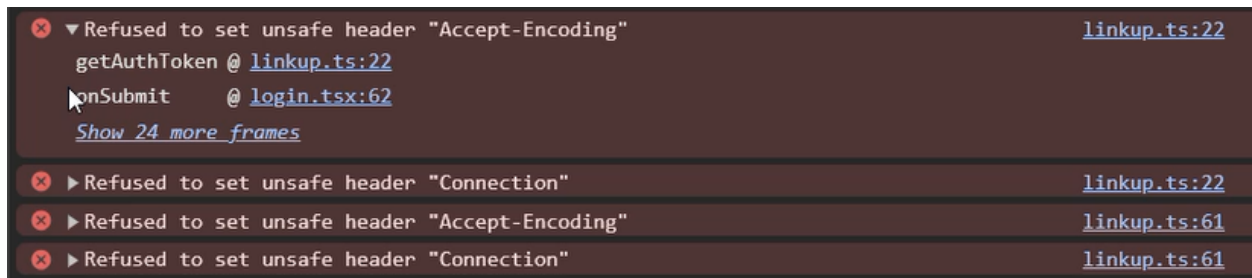
Stage 01

1. Summary

The Electron app crashes after a login attempt. Console logs show errors:

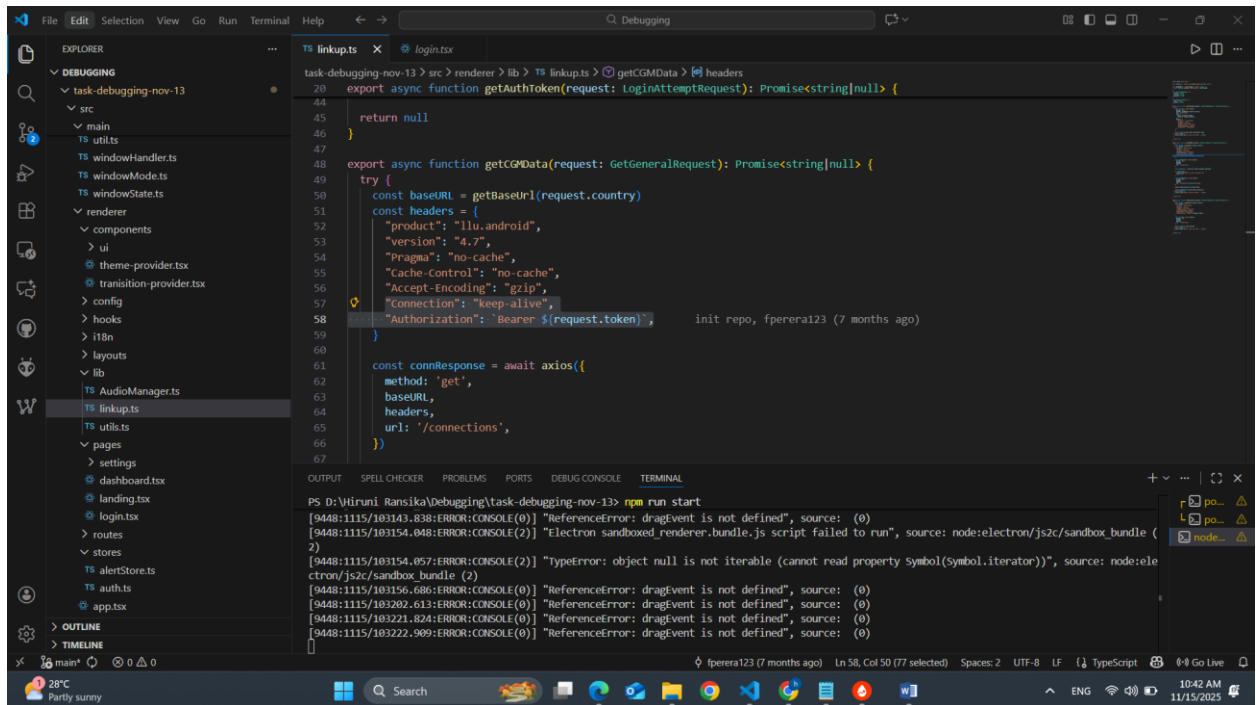
- Refused to set unsafe header "Accept-Encoding"
- Refused to set unsafe header "Connection"

Root cause: renderer code attempted to set browser-forbidden HTTP request headers. The request failed, returned `null`, and the UI proceeded with the missing token causing the crash.



2. Environment

- App: Electron (renderer + main processes)
- Network library: axios
- API: LibreView-like API (https://api-COUNTRY_CODE.libreview.io/llu)
- Files inspected: linkup.ts, login.tsx (renderer)

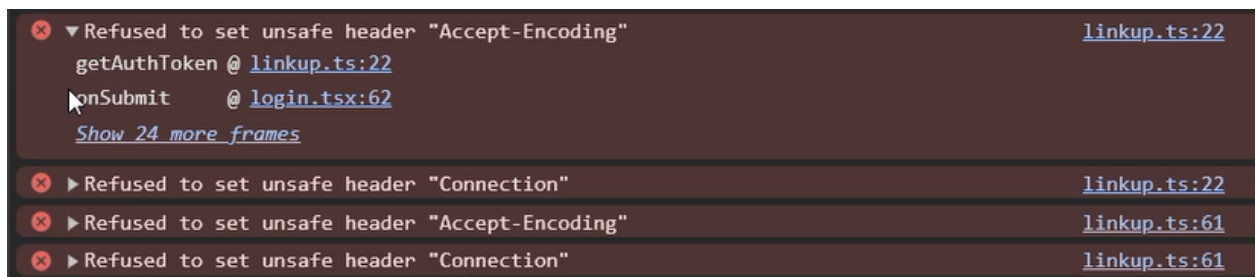


3. Reproducing the issue (steps)

1. Launch the Electron app (development mode).
2. Open app DevTools (View → Toggle Developer Tools).
3. Attempt to log in with valid credentials.
4. Observe console errors about refused headers and then app crash/hang.
5. Inspect network request — it does not succeed due to the thrown exception.

4. Observed errors

- Refused to set unsafe header "Accept-Encoding"
- Refused to set unsafe header "Connection"
- Stack trace points to linkup.ts:22 (the function that performs login).



5. Root cause

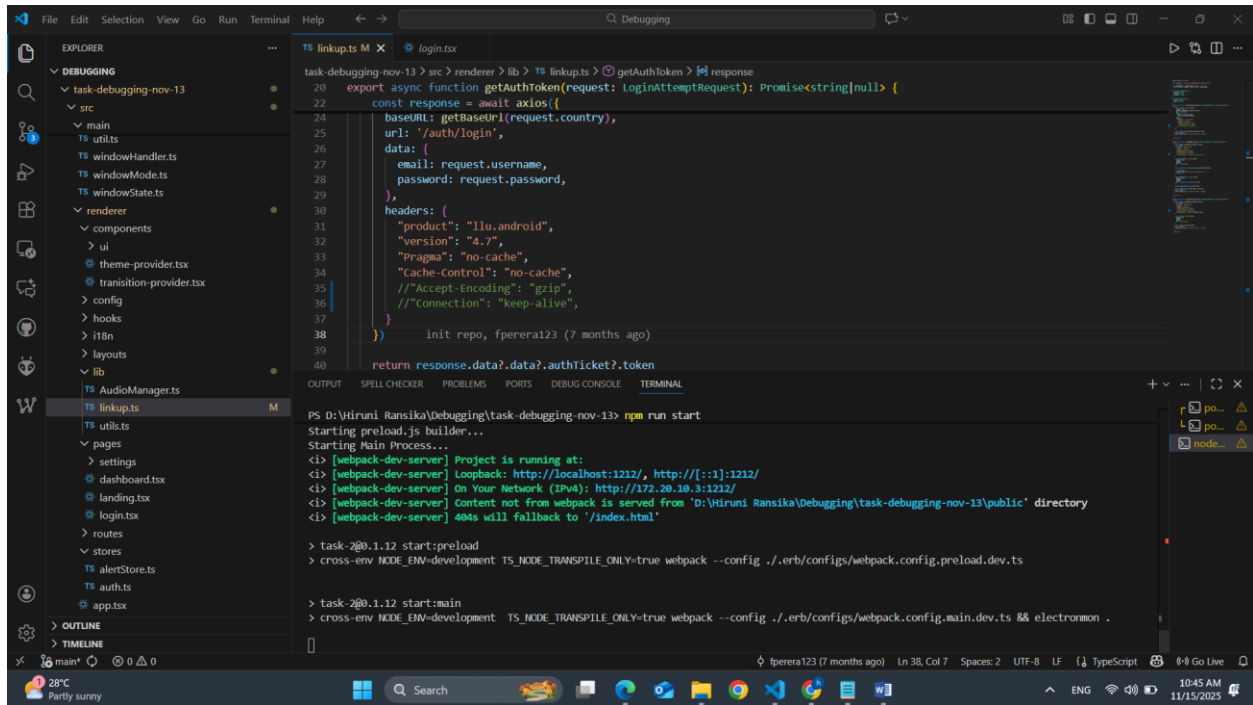
- The browser environment (including Electron renderer) forbids setting certain request headers from JavaScript: `Accept-Encoding`, `Connection`, `Content-Length`, `Host`, etc.
 - Attempting to set these headers throws an exception.
 - The login code didn't properly propagate or handle that failure (it returned `null`), and subsequent UI logic used the missing token causing a runtime crash.
-

6. Fixes implemented

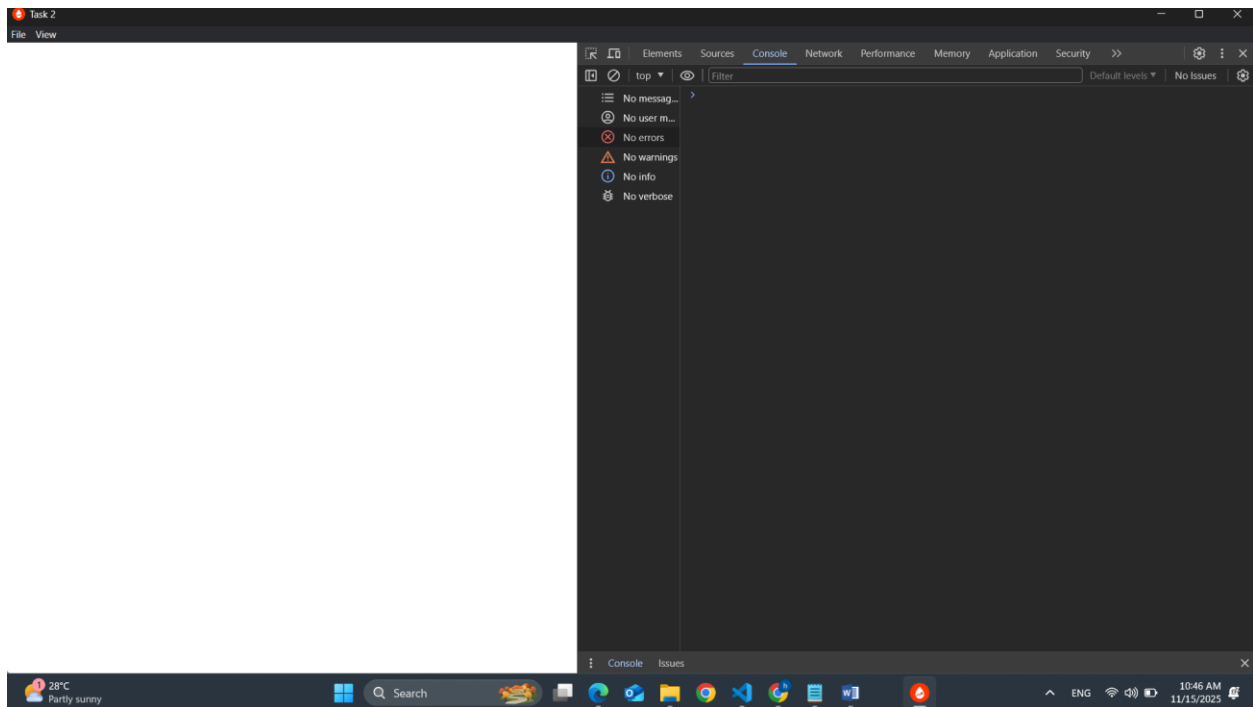
1. **Removed forbidden headers** from all axios requests:
 - Removed: `Accept-Encoding`, `Connection`.
 - Added/kept allowed headers like: `Accept: application/json`, `Content-Type` when needed, `Authorization: Bearer <token>`.
2. **Improved error handling:** functions return `null` on error and UI checks token existence before proceeding; errors logged and surfaced to user rather than causing uncaught exceptions.
3. **Sanity checks:** Validate API payloads before using fields (e.g., check `response.data?.data?.authTicket?.token` exists).

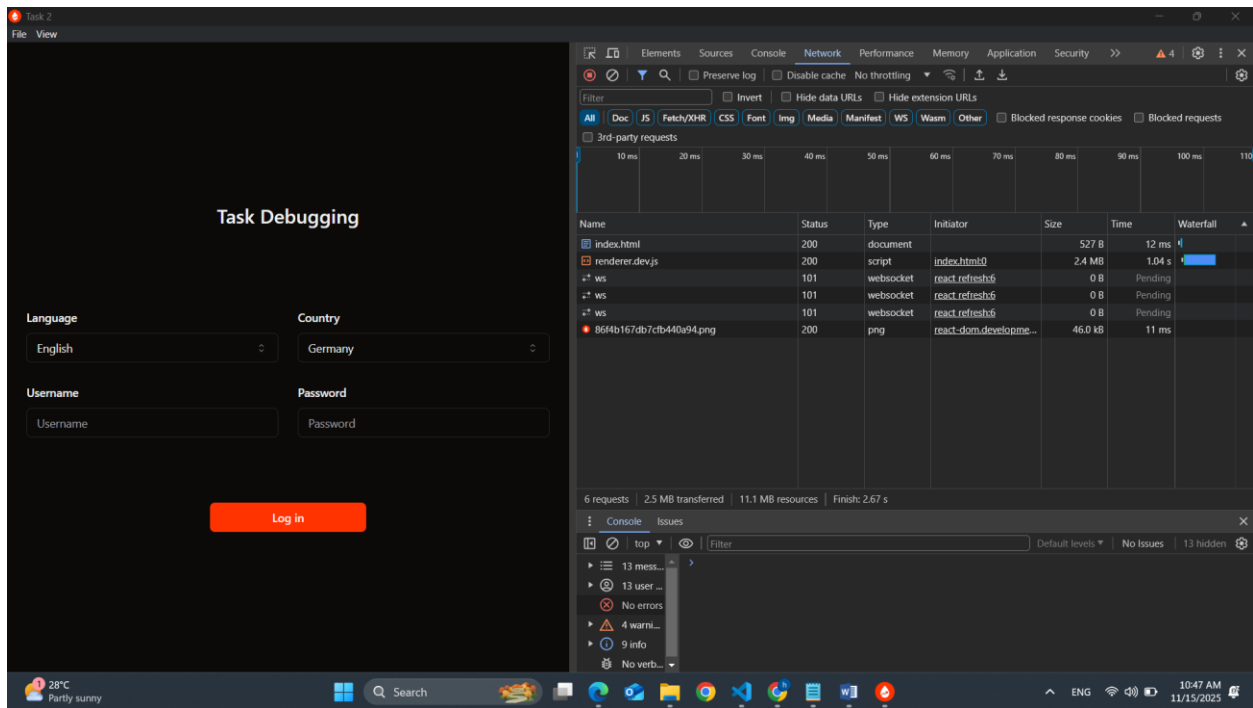
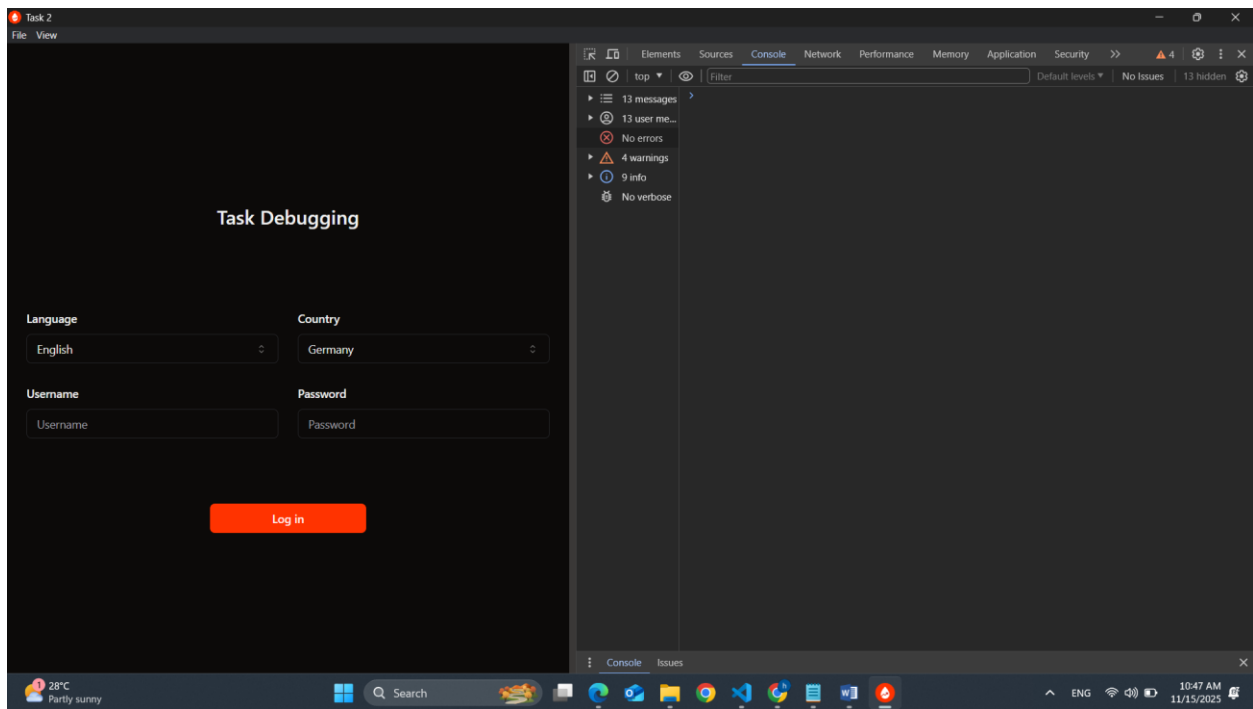
```
email: request.username,  
password: request.password,  
},  
headers: {  
  "product": "llu.android",  
  "version": "4.7",  
  "Pragma": "no-cache",  
  "Cache-Control": "no-cache",  
  // "Accept-Encoding": "gzip",  
  // "Connection": "keep-alive",  
}  
})  
init repo, fperera123 (7 months ago)  
  
return response.data?.data?.authTicket?.token  
} catch (error) {  
  console.log("Unable to get the token: ", error)  
}
```

Again run



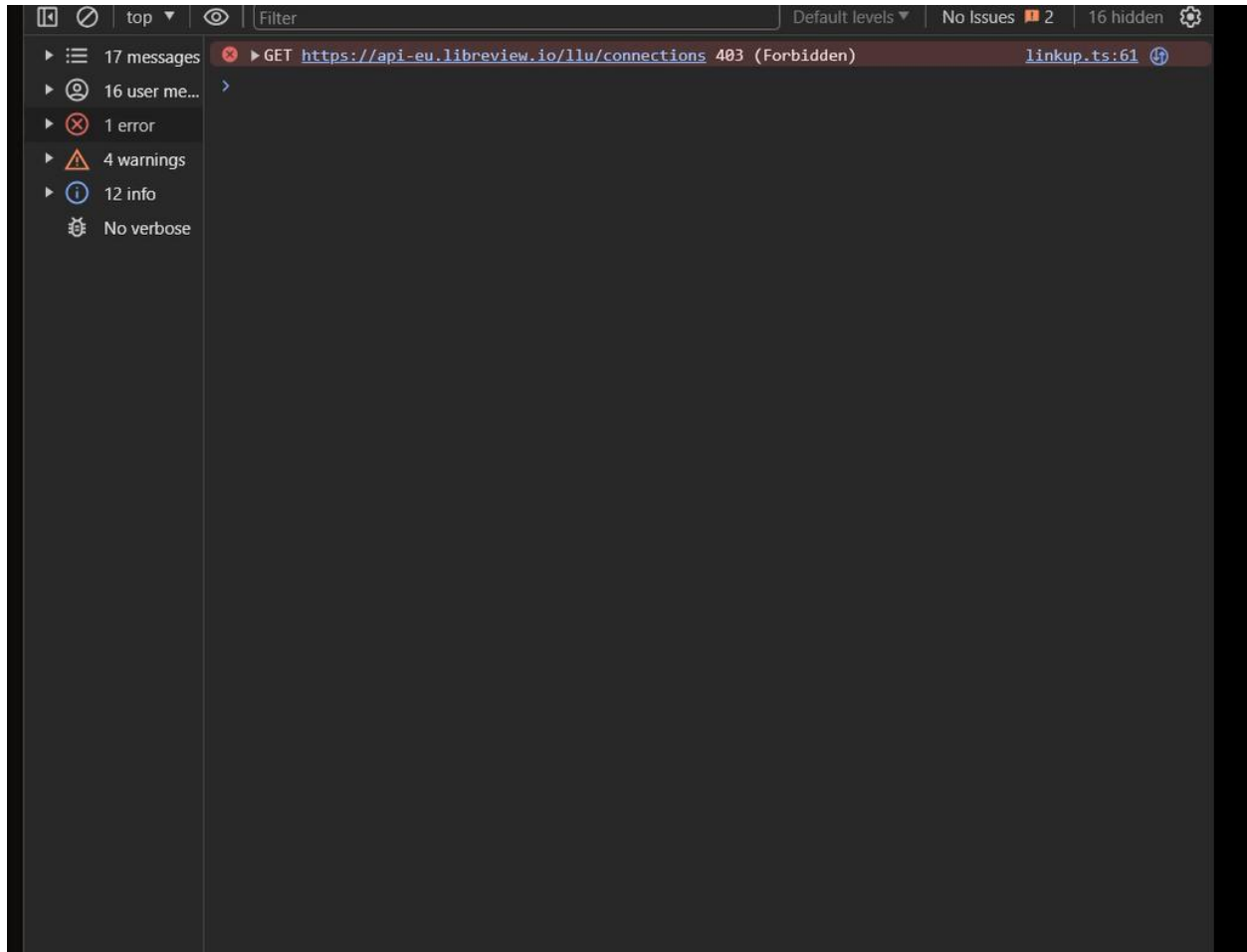
Then Solve previous error. But Still crashing.





Stage 02

Now I face new error



Now the error is:

```
GET https://api-eu.libreview.io/llu/connections 403 (Forbidden)
```

Your login succeeded
But your **token is NOT valid for /connections**, so the API rejects it.

This *exact error* means:

“You are logged in, but you are not authorized to access this user’s CGM data.”

Why you get 403 Forbidden? (3 possible reasons)

Wrong country code

Your base URL depends on this:

`https://api-COUNTRY_CODE.libreview.io/llu`

If your login uses:

`country: "ww", "eu", "us", "au", etc.`

But the user account belongs to **another region**, then:

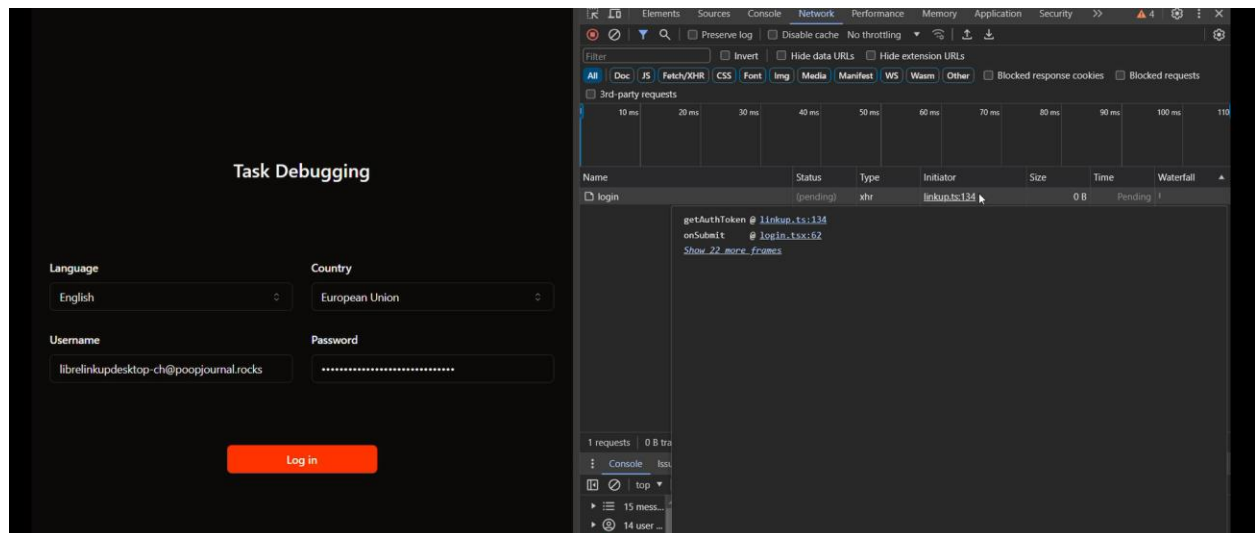
Login works

But CGM endpoints are region locked → 403 Forbidden.

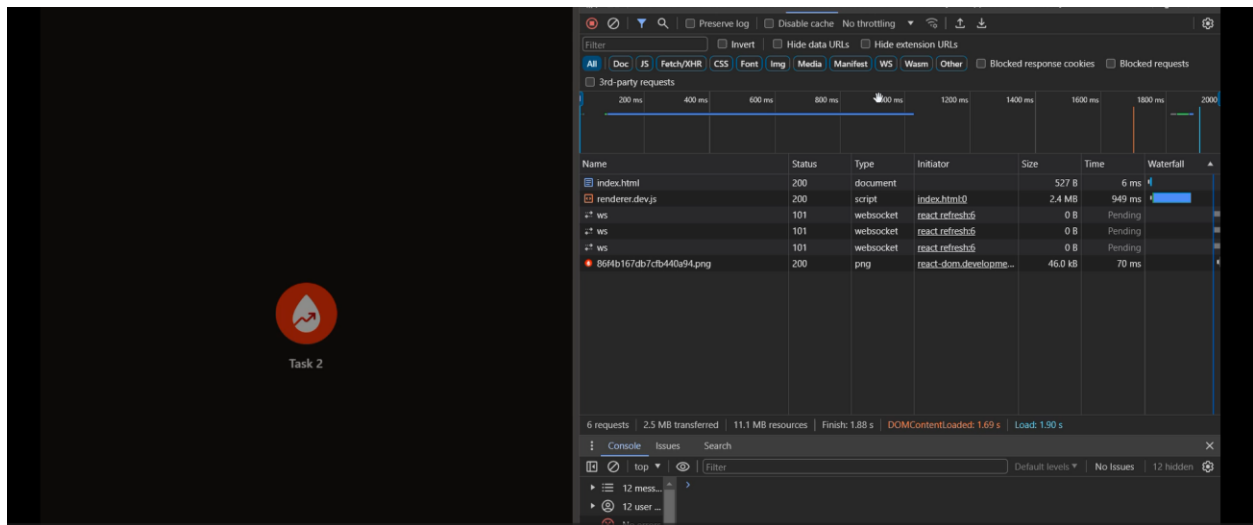
I do like that

```
const form = useForm<z.infer<typeof formSchema>>({
  resolver: zodResolver(formSchema),
  defaultValues: {
    username: "",
    password: "",
    country: "eu",
    language: "en",
  },
})
```

And again run

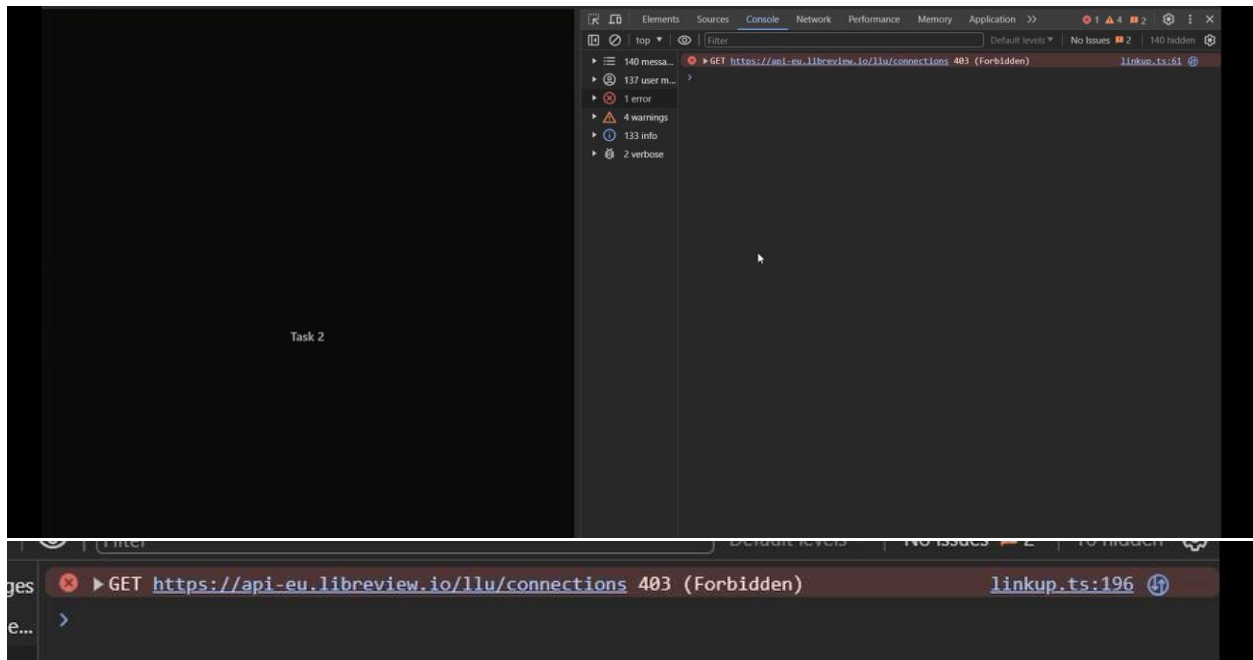


Again crash like that



I saw error appear and quickly disappear with error mark with connections and index.html

And in the console I see that error



GET https://api-eu.libreview.io/llu/connections 403 (Forbidden)

linkup.ts:61

codes

linkup.ts

```
import axios from "axios"

const BASE_URL = 'https://api-COUNTRY_CODE.libreview.io/llu'

const getBaseUrl = (countryCode: string): string => {
  return BASE_URL.replace('COUNTRY_CODE', countryCode)
}

type LoginAttemptRequest = {
  country: string
  username: string
  password: string
}

type GetGeneralRequest = {
  token: string
  country: string
}

export async function getAuthToken(request: LoginAttemptRequest):
Promise<string|null> {
  try {
    const response = await axios({
      method: 'post',
      baseUrl: getBaseUrl(request.country),
      url: '/auth/login',
      data: {
        email: request.username,
        password: request.password,
      },
      headers: {
        "product": "llu.android",
        "version": "4.7",
        "Pragma": "no-cache",
        "Cache-Control": "no-cache",
        //"Accept-Encoding": "gzip",
        //"Connection": "keep-alive",
      }
    })
    return response.data?.data?.authTicket?.token
  } catch (error) {
```

```

        console.log("Unable to get the token: ", error)
    }

    return null
}

export async function getCGMData(request: GetGeneralRequest):
Promise<string|null> {
    try {
        const baseUrl = getBaseUrl(request.country)
        const headers = {
            "product": "llu.android",
            "version": "4.7",
            "Pragma": "no-cache",
            "Cache-Control": "no-cache",
            //"Accept-Encoding": "gzip",
            //"Connection": "keep-alive",
            "Authorization": `Bearer ${request.token}`,
        }

        const connResponse = await axios({
            method: 'get',
            baseUrl,
            headers,
            url: '/connections',
        })

        const patientId = connResponse.data?.data[0]?.patientId

        if (!patientId) {
            console.log("Unable to get the patient id")
            return null
        }

        const graphResponse = await axios({
            method: 'get',
            baseUrl,
            headers,
            url: `/connections/${patientId}/graph`,
        })

        console.log(graphResponse?.data?.data)

        return graphResponse?.data?.data?.connection
    } catch (error) {

```

```

        console.log("Unable to get the token: ", error)
    }

    return null
}

export async function getConnection(request: GetGeneralRequest):
Promise<string|null> {
    try {
        const baseUrl = getBaseUrl(request.country)
        const headers = {
            "product": "llu.android",
            "version": "4.7",
            "Pragma": "no-cache",
            "Cache-Control": "no-cache",
            //"Accept-Encoding": "gzip",
            //"Connection": "keep-alive",
            "Authorization": `Bearer ${request.token}`,
        }

        const response = await axios({
            method: 'get',
            baseUrl,
            headers,
            url: '/connections',
        })

        return response?.data?.data[0]
    } catch (error) {
        console.log("Unable to get the token: ", error)
    }

    return null
}

```

Login

```

import * as z from "zod"
import { toast } from "sonner"
import { useTranslation, Trans } from "react-i18next"
import { useForm } from "react-hook-form"
import { useNavigate } from "react-router-dom"
import { zodResolver } from "@hookform/resolvers/zod"

```

```

import { PublicLayout } from "@layouts/public-layout"
import {
  Form,
  FormControl,
  FormField,
  FormItem,
  FormLabel,
  FormMessage,
} from "@components/ui/form"
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@components/ui/select"
import { Button } from "@components/ui/button"
import { Input } from "@components/ui/input"
import { getAuthToken } from "@lib/linkup"
import { countries, languages } from "@config/app"
import { useAuthStore } from "@stores/auth"
import { ButtonPopover } from "@components/ui/button-popover"
import { QuestionMarkIcon } from "@radix-ui/react-icons"
import logo from "../../assets/logo.png"
import { useEffect } from 'react';
import {
  getLocalStorageWindowMode,
  setWindowMode,
} from '@lib/utils';

const formSchema = z.object({
  username: z.string().min(2).max(50),
  password: z.string().min(2).max(50),
  country: z.string().min(2).max(50),
  language: z.string().min(2).max(50),
})

export default function LoginPage() {
  const navigate = useNavigate()
  const { i18n, t } = useTranslation()
  const setLanguage = useAuthStore((state) => state.setLanguage)

  const form = useForm<z.infer<typeof formSchema>>({
    resolver: zodResolver(formSchema),
    defaultValues: {

```

```

        username: "",
        password: "",
        country: "eu",
        language: "en",
    },
})

const login = useAuthStore((state) => state.login)

const onSubmit = async (values: z.infer<typeof formSchema>) => {
    const authToken = await getAuthToken({
        country: values.country,
        username: values.username,
        password: values.password,
    })

    if (authToken) {
        login(authToken, values.country, values.language)
        navigate('/dashboard')
    } else {
        toast.error("Invalid credentials.")
    }
}

const setAndRefreshLanguage = (l: string) => {
    i18n.changeLanguage(l)
    setLanguage(l)
}

// 🔄 overlay mode
// if it is in overlay mode automatically switch to windowed mode in this page
useEffect(() => {
    const fetchWindowMode = async () => {
        const mode = await getLocalStorageWindowMode();

        if(mode !== 'windowed'){
            setWindowMode('windowed');
        }
    };

    fetchWindowMode();
}, []);

return (
    <PublicLayout

```

```

        className="flex flex-col justify-center items-center"
    >
    <div className="flex flex-col items-center p-12">
        <h2 className="font-semibold text-2xl">Task Debugging</h2>
    </div>
    <Form {...form}>
        <form
            onSubmit={form.handleSubmit(onSubmit)}
            className="space-y-8 w-full p-6"
        >
            <div className="flex flex-col justify-between ">
                <div className="grid grid-cols-2 gap-6 mt-6">
                    <FormField
                        control={form.control}
                        name="language"
                        render={({ field }) => (
                            <FormItem>
                                <FormLabel>{ t('Language')}</FormLabel>
                                <FormControl>
                                    <Select onChange={field.onChange}
                                        {field.onChange(e); setAndRefreshLanguage(e) }}
                                        defaultValue={field.value}>
                                        <SelectTrigger>
                                            <SelectValue placeholder={t('SelectLanguage')} />
                                        </SelectTrigger>
                                        <SelectContent>
                                            {languages.map(language => (
                                                <SelectItem value={language.value}
                                                    key={language.value}>
                                                    {t(language.label)}
                                                </SelectItem>
                                            ))}
                                        </SelectContent>
                                    </Select>
                                </FormControl>
                            </FormItem>
                        )}
                    </div>
                    <FormField
                        control={form.control}
                        name="country"
                        render={({ field }) => (
                            <FormItem>
                                <FormLabel>{ t('Country') }</FormLabel>
                                <FormControl>

```

```

        <Select onChange={field.onChange}
defaultvalue={field.value}>
        <SelectTrigger>
        <SelectValue placeholder={ t('SelectCountry') } />
        </SelectTrigger>
        <SelectContent>
        {countries.map(country => (
        <SelectItem value={country.value}
key={country.value}>
        { t(country.label) }
        </SelectItem>
        ))}
        </SelectContent>
        </Select>
        </FormControl>
        </FormItem>
    )}
/>
<FormField
control={form.control}
name="username"
render={({ field }) => (
    <FormItem>
        <FormLabel>{ t('Username')}</FormLabel>
        <FormControl>
            <Input placeholder={ t('Username') } {...field} />
        </FormControl>
        <FormMessage />
    </FormItem>
)}
/>
<FormField
control={form.control}
name="password"
render={({ field }) => (
    <FormItem>
        <FormLabel>{ t('Password') }</FormLabel>
        <FormControl>
            <Input type="password" placeholder={ t('Password') }
{...field} />
        </FormControl>
        <FormMessage />
    </FormItem>
)}
/>

```

```

        </div>
        <div className="flex justify-center mt-20">
          <Button type="submit" className="w-48">{ t('Login') }</Button>
        </div>
      </div>
    </form>
  </Form>
</PublicLayout>
)
}

```

Dashboard

```

import { useEffect, useState } from 'react';
import { BaseLayout } from '@layouts/base-layout';
import { useAuthStore } from '@stores/auth';
import { getCGMDData } from '@lib/linkup';
import { TrendArrow } from '@components/ui/trend-arrow';
import {
  EnterFullScreenIcon,
  GearIcon,
} from '@radix-ui/react-icons';
import { useNavigate } from 'react-router-dom';
import { LoadingScreen } from '@components/ui/loading';
import { useClearSession } from '@hooks/session';
import {
  openNewWindow,
  setRedirectTo,
  getUserValue,
  getUserUnit,
  getLocalStorageWindowMode,
  setWindowMode,
} from '@lib/utils';

const LOW = 70;
const HIGH = 240;

export default function DashboardPage() {
  const { clearSession } = useClearSession();
  const navigate = useNavigate();
  const token = useAuthStore((state) => state.token);
  const country = useAuthStore((state) => state.country);
  const [graphData, setGraphData] = useState({});
  const [isReady, setIsReady] = useState(false);

```



```
const populateGraphData = async () => {
  const data = await getCGMData({
    token: token ?? '',
    country: country ?? '',
  });

  if (data === null) {
    clearSession();
    return;
  }

  setGraphData(data);
  setIsReady(true);
};

const getColor = (
  value: number,
  targetLow: number,
  targetHigh: number,
): string => {
  if (value < LOW) {
    return 'bg-red-500';
  }

  if (value > HIGH) {
    return 'bg-orange-500';
  }

  if (
    (value < targetLow && value >= LOW) ||
    (value > targetHigh && value <= HIGH)
  ) {
    return 'bg-yellow-500';
  }

  return 'bg-green-500';
};

useEffect(() => {
  populateGraphData();

  const interval = setInterval(() => {
    populateGraphData();
  }, 1000 * 60);
}, 1000 * 60);
```

```

    return () => clearInterval(interval);
  }, []);

const openSettings = (path: string) => {
  setRedirectTo(path);
  openNewWindow(path, 1024, 768);
};

// 📌 overlay mode functions
const [currentWindowMode, setCurrentWindowMode] = useState<null | string>(
  null,
);
useEffect(() => {
  const fetchWindowMode = async () => {
    const mode = await getLocalStorageWindowMode();
    setCurrentWindowMode(mode);

    if (mode === 'overlayTransparent') {
      document.body.style.background = 'transparent';
    } else {
      document.body.style.background = '';
    }
  };
  fetchWindowMode();
}, []);

const changeToWindowedMode = () => {
  setWindowMode('windowed');
}

if (!isReady) {
  return <LoadingScreen />;
}

return (
  <BaseLayout
    className={`${
      currentWindowMode === 'overlayTransparent'
        ? 'transparent'
        : getColor(
            graphData?.glucoseMeasurement?.ValueInMgPerDl ?? 1,
            graphData?.targetLow ?? 1,
            graphData?.targetHigh ?? 1,
          )
    }`
  >

```

```

    } flex justify-center items-center draggable`}
  >
  {currentWindowMode == 'windowed' ? (
    <button
      onClick={() => openSettings('/settings/general')}
      className="absolute 2xs:top-5 2xs:right-5 right-0 top-0 outline-none
hover:bg-white/20 p-2 rounded-md transition-all no-draggable"
    >
      <GearIcon className="text-white 2xs:h-6 2xs:w-6 w-4 h-4" />
    </button>
  ) : (
    <button
      onClick={() => changeToWindowedMode()}
      className="absolute 2xs:top-2 2xs:right-2 md:top-5 md:right-5 right-0
top-0 outline-none hover:bg-white/20 p-2 rounded-md transition-all no-draggable"
    >
      <div className=''>
        {/* <EnterFullScreenIcon className="text-white 2xs:h-6 2xs:w-6 w-4 h-4
outline-1" /> */}
        {/* Use a custom svg icon to just to add a shadow to the icon */}
        <svg width="15" height="15" viewBox="0 0 15 15" fill="none"
xmlns="http://www.w3.org/2000/svg" className="text-white 2xs:h-6 2xs:w-6 w-4 h-
4">
          <defs>
            <filter id="pathShadow" x="-50%" y="-50%" width="200%"
height="200%">
              <feDropShadow dx="2" dy="1" stdDeviation="2" flood-color="rgba(0,
0, 0, 0.9)"></feDropShadow>
            </filter>
          </defs>

          <path d="M2 2.5C2 2.22386 2.22386 2 2.5 2H5.5C5.77614 2 6 2.22386 6
2.5C6 2.77614 5.77614 3 5.5 3H3V5.5C3 5.77614 2.77614 6 2.5 6C2.22386 6 2 5.77614
2 5.5V2.5ZM9 2.5C9 2.22386 9.22386 2 9.5 2H12.5C12.7761 2 13 2.22386 13
2.5V5.5C13 5.77614 12.7761 6 12.5 6C12.2239 6 12 5.77614 12 5.5V3H9.5C9.22386 3 9
2.77614 9 2.5ZM2.5 9C2.77614 9 3 9.22386 3 9.5V12H5.5C5.77614 12 6 12.2239 6
12.5C6 12.7761 5.77614 13 5.5 13H2.5C2.22386 13 2 12.7761 2 12.5V9.5C2 9.22386
2.22386 9 2.5 9ZM12.5 9C12.7761 9 13 9.22386 13 9.5V12.5C13 12.7761 12.7761 13
12.5 13H9.5C9.22386 13 9 12.7761 9 12.5C9 12.2239 9.22386 12 9.5 12H12V9.5C12
9.22386 12.2239 9 12.5 9Z" fill="currentColor" fill-rule="evenodd" clip-
rule="evenodd" filter="url(#pathShadow)"></path>
        </svg>
      </div>
    </button>
  )}

```

```

<div className="flex items-center gap-3">
  <p
    className={`${
      currentWindowMode === 'overlayTransparent'
        ? 'overlay-shadow'
        : ''
    } text-white font-semibold xs:text-3xl text-xl`}
  >
    {getUserValue(graphData?.glucoseMeasurement?.ValueInMgPerDl) +
      ' ' +
      getUserUnit()}
  </p>
  <div
    className={`flex justify-center items-center xs:h-12 xs:w-12 h-6 w-6
rounded-full ${
      currentWindowMode === 'overlayTransparent'
        ? getColor(
            graphData?.glucoseMeasurement?.ValueInMgPerDl ?? 1,
            graphData?.targetLow ?? 1,
            graphData?.targetHigh ?? 1,
          )
        : 'bg-white/25'
    }`}
  >
    <TrendArrow
      className="h-9 w-9 text-white"
      trend={graphData?.glucoseMeasurement?.TrendArrow ?? 1}
    />
  </div>
</div>
</BaseLayout>
);
}

```

This Electron app crashes after login attempt. The crash seems related to:

1. API response after authentication
2. API usage

So, I 'm still find the solution. If I found new method or debug that. I will pull the request in github